

Stage DevOps — été, 2 mois

Stage sur une VM Linux : tu construis une petite plateforme conteneurisée (Kubernetes) autour d'indicateurs, avec un service IA en local.

Chaque semaine a un fichier dans `docs/besoins/`. Lis-le en entier avant de commencer.

Git — dès le départ

1. Clone le dépôt sur ta VM.
2. Crée ta branche personnelle et travaille toujours dessus :
3. `git checkout -b stagiaire/ton-prenom`
4. Commit et push régulièrement sur ta branche (pas sur `main`).
5. Tes comptes-rendus de semaine vont dans `livraisons/semaine-XX/`.

Le but du stage

À la fin (semaine 8), j'ouvre ton interface dans le navigateur et je dois voir :

1. Les indicateurs — stockés en base PostgreSQL, affichés à l'écran.
2. Un workflow de validation — on peut marquer certaines valeurs comme validées (comment tu le fais, tu le conçois).
3. Une réponse IA — générée par un modèle gratuit installé en local, uniquement à partir des indicateurs validés.

L'architecture (services, conteneurs, routes, tables...) c'est toi qui la proposes en semaine 1. Je ne te donne pas de schéma tout fait. Détails : [architecture.md](#).

PostgreSQL ne va jamais dans un conteneur — c'est un serveur à part.

Planning

Sem	Sujet	Fichier
1	Architecture + squelette	semaine-01
2	Frontend Angular (données statiques)	semaine-02
3	Gateway	semaine-03
4	Service métier + Postgres	semaine-04
5	Service IA + modèle local	semaine-05
6	Frontend branché aux APIs	semaine-06
7	IA visible dans l'interface	semaine-07
8	Tout ensemble + validation	semaine-08

L'ordre des semaines 2 à 8 suit la logique classique (front → gateway → métier → IA → branchement → finition), mais les détails techniques viennent de ton architecture validée en S1.

Semaine 1 — Architecture & mise en place

Contexte

Avant de coder quoi que ce soit, tu dois avoir une vision claire du projet. Les deux premiers jours servent à ça.

Ce que tu fais

Jours 1–2 : conception (à me présenter)

Prépare un document avec :

1. Schéma d'architecture — dessine tes services (conteneurs), montre qui communique avec qui (flèches, ports, noms de services K8s). Exemple de questions à trancher : le front passe-t-il par un gateway ? l'IA peut-elle toucher la base directement ou passe-t-elle par le métier ?
2. Modèle de données — 3 ou 4 tables pour les indicateurs. Pour chaque table : nom, champs principaux, clés, liens. Pense déjà au statut de validation (ex. `brouillon` / `validé`).
3. Choix du modèle IA — compare 2 ou 3 modèles open source gratuits utilisables en local (Mistral, Llama, Phi...). Dis lequel tu retiens, pourquoi (RAM, vitesse, licence), et comment tu comptes l'installer.

Après ma validation

4. Squelette du projet — crée l'arborescence (`services/`, `k8s/`, `livraisons/`, etc.) sur ta branche Git.

5. Environnement VM — installe Git, Docker, un Kubernetes local (Minikube, k3s...). Vérifie que la VM joint le serveur Postgres (port 5432).

Git

```
git checkout -b stagiaire/ton-prenom  
git push -u origin stagiaire/ton-prenom
```

Tout ton travail du stage se fait sur cette branche.

Résultat attendu

- Un schéma d'architecture validé par le tuteur.
- Un modèle de données clair.
- Un modèle IA choisi et justifié.
- La VM prête, le repo structuré, la branche créée.

À rendre

livraisons/semaine-01/README.md — schéma, tables, choix IA, état de la VM.

Semaine 2 — Frontend Angular (statique)

Contexte

Premier vrai conteneur sur Kubernetes. Tu construis l'interface que l'utilisateur verra à la fin — mais pour l'instant avec des fausses données, sans backend.

Ce que tu fais

1. Crée une app Angular avec au minimum :
 - une page liste des indicateurs (nom, valeur, ce que tu as prévu dans tes tables S1) ;
 - une page détail quand on clique sur un indicateur.
2. Les données sont en dur dans le code ou dans un fichier JSON local — pas d'appel API.
3. Conteneurise l'app (Dockerfile) et déploie sur Kubernetes.
4. L'interface doit être accessible dans le navigateur (NodePort, Ingress... ton choix).

Pourquoi en statique d'abord ?

Ça te permet de valider le design des écrans et le déploiement K8s avant de brancher les vraies APIs.

Résultat attendu

Je tape une URL, je vois la liste des indicateurs et je peux ouvrir un détail. Tout est statique mais conforme à ton modèle de données S1.

À rendre

livraisons/semaine-02/README.md — URL d'accès, comment builder/déployer, capture d'écran liste + détail.

Commit sur ta branche `stagiaire/ton-prenom`.

Semaine 3 —

Gateway

Contexte

Le gateway est le point d'entrée pour les appels API. Le frontend (et plus tard l'extérieur) ne parle pas directement aux services internes — tout passe par lui, selon ce que tu as dessiné en S1.

Ce que tu fais

1. Crée un service Gateway (.NET 8) dans son propre conteneur.
2. Ajoute un endpoint `/health` qui répond OK (pour vérifier que le service tourne).
3. Configure les routes vers le futur service métier (ex. `/api/indicators` → `api-metier`). Si le métier n'existe pas encore, un mock ou une réponse 502 documentée suffit pour cette semaine.
4. Déploie sur K8s en plus du frontend S2 — les deux doivent tourner en même temps.

Résultat attendu

- `curl` sur `/health` → OK.
- `curl` sur une route métier → réponse (même mock) ou erreur explicite documentée.
- Le frontend S2 est toujours accessible.

À rendre

livraisons/semaine-03/README.md — schéma des routes, commandes `curl`, captures ou sorties terminal.

Commit sur ta branche.

Semaine 4 — Service métier + PostgreSQL

Contexte

C'est le cœur métier : les indicateurs vivent en base, et ton API permet de les créer, lire, modifier et supprimer (CRUD).

Ce que tu fais

1. Crée le service métier (.NET 8) dans un conteneur séparé.
2. Implémente le CRUD sur les indicateurs, en suivant les tables conçues en S1.
3. Connecte-le au PostgreSQL fourni par le tuteur :
 - chaîne de connexion dans un Secret Kubernetes ;
 - jamais de mot de passe dans Git.
4. Branche les routes du Gateway S3 vers ce service.
5. Depuis l'extérieur, on accède au métier uniquement via le Gateway — pas d'exposition directe du pod métier.

Infos Postgres (tuteur)

--	--

IP	à compléter
Port	5432
Base	à compléter
User	à compléter

Résultat attendu

Avec `curl` via le Gateway, je peux créer un indicateur, le lire, le modifier et le supprimer. Les données sont bien en base Postgres.

À rendre

[livraisons/semaine-04/README.md](#) — exemples `curl` pour chaque opération CRUD, schéma Gateway → Métier → Postgres.

Commit sur ta branche.

Semaine 5 —

Service IA

Contexte

Quatrième service : l'IA. Elle ne remplace pas le métier — elle s'appuie sur les données indicateurs pour produire une analyse ou une synthèse, via un modèle installé en local sur la VM.

Ce que tu fais

1. Crée le service IA (.NET 8) dans un conteneur isolé.
2. Pour récupérer les indicateurs, appelle le métier en HTTP interne (ex. `http://api-metier:8080/indicators`) — pas de connexion directe à PostgreSQL depuis l'IA (sauf si tu l'as justifié autrement en S1 et que c'est validé).
3. Installe et appelle le modèle IA local choisi en S1 (Ollama, llama.cpp...).
4. Expose au moins un endpoint via le Gateway (ex. analyse, synthèse).
5. Le modèle reçoit les données indicateurs et génère une réponse texte.

Résultat attendu

Un `curl` via le Gateway vers ton endpoint IA renvoie une réponse cohérente, basée sur de vraies données en base (pas du texte inventé sans lien avec les indicateurs).

À rendre

livraisons/semaine-05/README.md — schéma du flux (IA → métier → modèle local), commande `curl`, exemple de réponse.

Commit sur ta branche.

Semaine 6 — Frontend branché aux APIs

Contexte

Jusqu'ici le front affichait des données en dur. Cette semaine tu le connectes au vrai backend via le Gateway.

Ce que tu fais

1. Remplace les données statiques de la S2 par des appels HTTP au métier (via le Gateway).
2. La liste et le détail des indicateurs viennent de la base Postgres en temps réel.
3. Garde les mêmes écrans autant que possible — tu changes la source de données, pas forcément tout le design.
4. Gère au minimum le cas où l'API ne répond pas (message d'erreur, chargement...).

Résultat attendu

J'ouvre l'interface : les indicateurs affichés sont ceux en base. Si j'en crée un via `curl`, il apparaît dans le front après rafraîchissement.

À rendre

`livraisons/semaine-06/README.md` — captures montrant des données live, comment le front appelle le Gateway.

Commit sur ta branche.

Semaine 7 — IA dans l'interface

Contexte

Le service IA tourne (S5) mais l'utilisateur ne le voit pas encore. Cette semaine tu l'intègres dans l'interface Angular.

Ce que tu fais

1. Depuis le front, appelle le service IA via le Gateway.
2. Affiche la réponse générée à l'écran (zone de texte, panneau, modal... à toi de voir).
3. L'utilisateur doit pouvoir déclencher l'analyse (bouton, action au chargement...).
4. Corrige les bugs ou points faibles repérés jusqu'ici (logs, erreurs, doc de déploiement...).

Résultat attendu

Dans le navigateur : indicateurs visibles + un bouton ou action qui affiche la réponse IA.

À rendre

livraisons/semaine-07/README.md — capture avec indicateurs + réponse IA visible.

Commit sur ta branche.

Semaine 8 — Validation & plateforme complète

Contexte

Dernière semaine : tout doit fonctionner ensemble, avec une règle métier importante — l'IA ne travaille que sur les indicateurs validés.

Ce que tu fais

1. Workflow de validation des valeurs

Conçois et implémente un parcours pour qu'une valeur d'indicateur puisse être validée ou non validée. C'est toi qui décides du détail :

- un champ en base (`statut, is_validated...`) ;

- un bouton dans l'interface ;
- une API dédiée (`PATCH /indicators/{id}/validate`) ;
- plusieurs étapes (brouillon → en revue → validé) si tu veux.

L'important : on doit pouvoir distinguer clairement les indicateurs validés des autres.

2. Filtrage côté IA

Quand l'IA génère une réponse, elle ne reçoit et n'analyse que les indicateurs dont la valeur est validée. Les autres sont ignorés.

Exemple : 5 indicateurs en base, 2 validés → la réponse IA ne parle que de ces 2-là.

3. Démo bout en bout

Tous les conteneurs tournent. Depuis le navigateur :

1. Je vois les indicateurs (base Postgres).
2. Je peux valider certaines valeurs (ton workflow).
3. Je lance l'IA → la réponse ne porte que sur les indicateurs validés.

Résultat attendu

La plateforme complète est stable. Le flux validation → IA filtrée est démontrable en live.

À rendre

livraisons/semaine-08/README.md contenant :

- URL de démo ;
- schéma final de ton architecture ;
- explication de ton workflow de validation (étapes, statuts, qui fait quoi) ;
- capture ou scénario : indicateurs validés vs non validés + réponse IA correspondante.

Dernier push sur ta branche `stagiaire/ton-prenom`.

Architecture

Il n'y a pas d'architecture imposée. C'est ton travail de la proposer en semaine 1.

Ce que tu dois définir

- Les services : combien de conteneurs, à quoi ils servent (front, gateway, métier, IA...).
- Les échanges : qui appelle qui, par quel protocole, quelles URLs internes K8s.
- La base de données : 3 à 4 tables autour des indicateurs (structure, relations).
- Le modèle IA : quel modèle open source gratuit tu feras tourner en local, et comment le service IA s'y connecte.
- Le workflow de validation (à prévoir dès la conception) : comment une valeur d'indicateur passe de « saisie » à « validée ».

Tu me présentes tout ça les 2 premiers jours. On valide ensemble avant que tu codes.

Ce que je fournis

- Une VM Linux pour travailler.
- Un serveur PostgreSQL externe (pas dans Kubernetes).

Résultat attendu en semaine 8

```
Utilisateur → interface web
                → indicateurs lus en base
                → validation des valeurs (ton workflow)
                → IA qui analyse seulement les indicateurs validés
```

Le chemin technique pour y arriver, c'est ton schéma.

Git

Travaille sur ta branche `stagiaire/ton-prenom`. Documente l'architecture validée dans `livraisons/semaine-01/`.